

Machine Learning — Part 4

Mathematical Derivations, Worked Examples & Python Implementations

Scope: Model tuning → K-fold cross-validation → Grid/Random Search → Ensemble Learning → Stacking → Bagging → Random Forest → Boosting → AdaBoost → Gradient Boosting → XGBoost → Unsupervised Learning → K-means → Elbow Method → Hierarchical Clustering → DBSCAN → Dimensionality Reduction → PCA.

How to use this guide: Every major concept is explained in four layers: intuition, mathematical objective, derivation/worked example, and a compact Python implementation. Where the source transcript is conceptual rather than mathematical, this guide supplies a small self-contained dataset and derives the mathematics from first principles.

Source basis: the uploaded Part 4 transcript. The transcript explicitly covers model tuning, K-fold cross-validation, grid/random search, ensemble methods, bagging/boosting, random forest, and unsupervised methods including K-means, DBSCAN and PCA.

Study-guide edition • August 2026

Table of Contents

1. 1. Mathematical foundations used throughout
2. 2. Model tuning and hyperparameters
3. 3. K-fold cross-validation
4. 4. Grid Search CV
5. 5. Randomized Search CV
6. 6. Ensemble learning
7. 7. Stacking
8. 8. Bagging and Random Forest
9. 9. Boosting
10. 10. AdaBoost
11. 11. Gradient Boosting
12. 12. XGBoost
13. 13. Unsupervised learning
14. 14. K-means clustering
15. 15. Elbow method and choosing K
16. 16. Hierarchical clustering
17. 17. DBSCAN
18. 18. Dimensionality reduction
19. 19. PCA — full derivation
20. 20. Integrated worked mini-project
21. 21. Formula sheet and exam checklist

1. Mathematical Foundations Used Throughout

The later algorithms repeatedly use the same mathematical building blocks: vectors, matrices, distances, averages, variance, covariance, optimization, probability and loss functions.

1.1 Vectors and matrices

A data point with d features is a vector $x = [x_1, x_2, \dots, x_d]^T$. A dataset of n observations is an $n \times d$ matrix X .

For example, if each customer has age and annual spend, $x = [25, 40]$ means age = 25 and spend = 40 (after whatever units/scaling are being used).

1.2 Mean, variance and standard deviation

Mean: $\mu = (1/n) \sum_{i=1}^n x_i$

Population variance: $\sigma^2 = (1/n) \sum_{i=1}^n (x_i - \mu)^2$

Sample variance: $s^2 = (1/(n-1)) \sum_{i=1}^n (x_i - \bar{x})^2$

Standard deviation: $\sigma = \sqrt{\sigma^2}$

Worked example. For $x = [2, 4, 6]$, $\mu = 4$. Squared deviations are 4, 0, 4, so population variance = $8/3 \approx 2.667$ and $\sigma \approx 1.633$.

1.3 Euclidean and Manhattan distance

Euclidean distance: $d(x,y) = \sqrt{\sum_j (x_j - y_j)^2}$

Manhattan distance: $d(x,y) = \sum_j |x_j - y_j|$

Worked example. For $x=(2,3)$ and $y=(5,7)$: Euclidean = $\sqrt{3^2+4^2}=5$; Manhattan = $3+4=7$. KNN and clustering can change their behavior depending on the distance metric.

1.4 Standardization

$z = (x - \mu)/\sigma$

Standardization centers a feature at 0 with unit standard deviation. It is especially important for distance- and margin-based methods such as KNN, SVM and K-means. The source transcript also standardizes X before SVM cross-validation and before clustering.

Worked example. If a feature has $\mu=50$, $\sigma=10$ and $x=70$, then $z=(70-50)/10=2$. The point is two standard deviations above the mean.

1.5 Optimization language

General learning problem: $\theta^* = \operatorname{argmin}_{\theta} J(\theta)$

θ denotes model parameters learned from data; J is an objective/loss. Hyperparameters are chosen outside this optimization loop by validation. This distinction is central to model tuning.

2. Model Tuning and Hyperparameters

The source emphasizes that a first model is not necessarily the best model and that hyperparameters control model behavior. Examples given include K in KNN, tree depth, learning rate and kernel choice.

`filecite[turn0file0]L37-L45`

A useful distinction: parameters are learned from the training data (e.g., linear-regression coefficients); hyperparameters are selected before/during model selection (e.g., K in KNN, `max_depth` in a tree, C and γ in an SVM).

2.1 KNN: mathematical prediction

For query x , find $N_k(x)$, the k nearest training points.

Unweighted classification: $\hat{y} = \text{mode}\{y_i : x_i \in N_k(x)\}$

Distance-weighted voting: $\text{vote}(c) = \sum_{i \in N_k(x)} 1[y_i=c] \cdot w_i$, where $w_i = 1/(d(x, x_i) + \epsilon)$

Worked example. Suppose the three nearest labels are [Red, Green, Green]. Green receives 2 votes, so prediction = Green. With $k=5$, suppose labels become [Red, Green, Green, Red, Red]; Red receives 3 votes and the prediction flips. This is exactly why K is a hyperparameter.

2.2 Regularization: Ridge and Lasso

Ridge: $J(\beta) = (1/n) \sum_i (y_i - x_i^T \beta)^2 + \lambda \sum_j \beta_j^2$

Lasso: $J(\beta) = (1/n) \sum_i (y_i - x_i^T \beta)^2 + \lambda \sum_j |\beta_j|$

Elastic Net: $J(\beta) = \text{MSE} + \lambda[\alpha \sum |\beta_j| + (1-\alpha) \sum \beta_j^2]$

Increasing λ penalizes large coefficients more strongly. L1 regularization can drive coefficients exactly to zero; L2 usually shrinks them toward zero. In model selection, λ (and Elastic Net α) can be treated as hyperparameters.

2.3 SVM hyperparameters

Soft-margin objective: minimize $(1/2) \|w\|^2 + C \sum_i \xi_i$ subject to $y_i(w^T x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$

Equivalent hinge-loss view: $L = (1/2) \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b))$

RBF kernel: $K(x, z) = \exp(-\gamma \|x - z\|^2)$

Typical SVM hyperparameters are C , kernel and γ (for RBF). The transcript explicitly discusses changing the SVM kernel and observing a different cross-validation score. `filecite[turn2file9]L154-L159`

3. K-Fold Cross-Validation

The transcript contrasts a single train/test split with K-fold cross-validation. It explains that each fold becomes the test portion once while the remaining folds are used for training, producing K scores whose average is reported. [filecite\turn1file7\L119-L126](#)

3.1 Mathematical setup

Dataset $D = \{(x_i, y_i)\}_{i=1}^n$ is partitioned into K disjoint folds $D_1, \dots, D_K$.

For fold k: train on $D \setminus D_k$ and validate on D_k .

CV score = $(1/K) \sum_{k=1}^K S(f_{-k}, D_k)$

Worked example. Take 10 observations and $K=5$. Each fold has 2 observations. Each run trains on 8 and validates on 2. If accuracies are 82%, 86%, 84%, 85%, 81%, then mean CV accuracy = $(82+86+84+85+81)/5 = 83.6\%$.

3.2 Why averaging helps

A single split can be unusually easy or difficult. Cross-validation samples multiple train/validation configurations, so the reported mean is less dependent on one arbitrary split. It also gives a dispersion estimate.

Sample standard deviation of CV scores: $s_{CV} = \sqrt{[(1/(K-1)) \sum_k (S_k - S)^2]}$

A high mean with high spread means the model is strong on average but unstable across folds.

3.3 Accuracy and other metrics

Accuracy = $(TP+TN)/(TP+TN+FP+FN)$

Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$

F1 = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$

MSE = $(1/n) \sum (y_i - \hat{y}_i)^2$, RMSE = $\sqrt{\text{MSE}}$

MAE = $(1/n) \sum |y_i - \hat{y}_i|$

3.4 Python

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC

model = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1, gamma="scale"))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

print(scores)
print(scores.mean())
print(scores.std())
```

4. Grid Search CV

Grid search enumerates a predefined set of hyperparameter combinations and evaluates each combination using cross-validation. The source explicitly introduces GridSearchCV after explaining why cross-validation is needed for tuning. [filecite\turn2file8\L137-L143](#)

4.1 Mathematical formulation

Let $H = H_1 \times H_2 \times \dots \times H_p$ be the Cartesian product of candidate hyperparameter values.

$|H| = \prod_{j=1}^p |H_j|$ candidate combinations.

For each $h \in H$: $CV(h) = (1/K) \sum_k S(f_{\{h,-k\}}, D_k)$.

$h^* = \operatorname{argmax}_{\{h \in H\}} CV(h)$ (or argmin for a loss).

Worked example. If $C \in \{0.1, 1, 10\}$ and $\text{kernel} \in \{\text{linear}, \text{rbf}\}$, the grid has $3 \times 2 = 6$ combinations. With 5-fold CV, there are 30 model fits (ignoring refitting on the full data).

4.2 Python

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

grid = {
    "C": [0.1, 1, 10],
    "kernel": ["linear", "rbf"],
    "gamma": ["scale", 0.1]
}

search = GridSearchCV(
    SVC(),
    param_grid=grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
search.fit(X, y)

print(search.best_params_)
print(search.best_score_)
```

The source explains that the results table contains mean test score for each parameter combination and that the objective is to identify the combination with the best average CV score. [filecite\turn2file13\L220-L226](#)

5. Randomized Search CV

Randomized search samples hyperparameter combinations rather than evaluating the entire Cartesian grid. The transcript describes it as similar to grid search but with a fixed number of iterations/samples.

filecite{turn2file6}{L106-L111}

5.1 Mathematical view

Let $\pi(h)$ be a probability distribution over hyperparameters. Sample $h_1, \dots, h_T \sim \pi(h)$.

$h^* = \operatorname{argmax}_{\{t=1, \dots, T\}} \operatorname{CV}(h_t)$.

Computational cost $\approx T \times K$ fits (instead of $|H| \times K$).

Worked example. Suppose a grid has 4 choices for C , 5 for γ and 3 kernels: $4 \times 5 \times 3 = 60$ combinations. With $K=5$, exhaustive grid search requires 300 fits. RandomizedSearchCV with $n_{\text{iter}}=12$ needs about 60 fits, so it can explore a large space much more cheaply.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform
from sklearn.svm import SVC

param_dist = {
    "C": loguniform(1e-2, 1e2),
    "gamma": loguniform(1e-4, 1e0),
    "kernel": ["rbf", "linear"]
}

search = RandomizedSearchCV(
    SVC(),
    param_distributions=param_dist,
    n_iter=12,
    cv=5,
    scoring="accuracy",
    random_state=42,
    n_jobs=-1
)
search.fit(X, y)
```

6. Ensemble Learning

Ensemble learning combines multiple models to obtain a stronger final predictor. The source presents stacking, bagging and boosting as the three major ensemble techniques discussed.

□filecite□turn2file11□L187-L192□

6.1 Voting

Hard voting classification: $\hat{y} = \text{mode}\{f_1(x), \dots, f_M(x)\}$

Soft voting: $\hat{y} = \text{argmax}_c \sum_m w_m P_m(y=c|x)$, with $w_m \geq 0$ and $\sum w_m = 1$

Regression averaging: $\hat{y} = (1/M) \sum_m f_m(x)$

Worked example. Three classifiers predict [1,0,1]. Majority vote is 1. If their probabilities for class 1 are 0.90, 0.40, 0.80, the mean is 0.70, so soft voting also predicts class 1.

6.2 Why ensembles can reduce error

If errors $e_1, \dots, e_M$ are unbiased with variance σ^2 and pairwise correlation ρ , the variance of the average is approximately $\sigma^2[1+(M-1)\rho]/M$.

If ρ is low, averaging many models substantially reduces variance. If all models make identical errors ($\rho \approx 1$), averaging gives little benefit. This is the mathematical reason diversity matters.

7. Stacking

Stacking trains several base learners and then trains a meta-learner on their predictions. The transcript gives an example where outputs from SVM, decision tree and logistic regression are fed to a KNN meta-model.

filecite turn2file1 L27-L32

7.1 Mathematical formulation

Base models: $z_m = f_m(x)$, $m=1, \dots, M$.

Meta-feature vector: $z = [f_1(x), f_2(x), \dots, f_M(x)]$.

Final prediction: $\hat{y} = g(z)$.

Critical point: To avoid leakage, meta-model training should use out-of-fold predictions rather than predictions from base models trained on the same rows. For K folds, every training observation receives a prediction from a model that did not train on that observation.

Worked example. With 200 training rows and 5 folds, each base learner predicts the held-out 40 rows in turn. Concatenating the five held-out prediction sets gives 200 out-of-fold predictions per base model. Three base models therefore create a 200×3 meta-feature matrix.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

estimators = [
    ("lr", LogisticRegression(max_iter=1000)),
    ("svm", SVC(probability=True)),
    ("tree", DecisionTreeClassifier(max_depth=3, random_state=42))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(),
    cv=5
)
stack.fit(X_train, y_train)
```

8. Bagging and Random Forest

Bagging means Bootstrap Aggregating: repeatedly sample training rows with replacement, fit a model to each bootstrap sample, then aggregate predictions. The transcript explicitly explains bootstrap sampling and links bagging to Random Forest. [filecite\turn1file9\L154-L158](#)

8.1 Bootstrap mathematics

For n training observations, draw n times with replacement.

Probability a particular observation is not selected in one draw $= 1 - 1/n$.

Probability it is never selected after n draws $= (1 - 1/n)^n \rightarrow e^{-1} \approx 0.368$.

Therefore expected unique observations in a bootstrap sample $\approx n(1 - e^{-1}) \approx 0.632n$.

Worked example. For $n=100$, about 36.8 observations are expected to be absent from a bootstrap sample, leaving about 63.2 unique observations. The omitted observations are called out-of-bag (OOB) examples.

8.2 Bagged predictor

Regression: $\hat{f}_{\text{bag}}(x) = (1/B) \sum_{b=1}^B f_b(x)$

Classification: $\hat{y} = \text{mode}\{f_1(x), \dots, f_B(x)\}$

8.3 Random Forest

Random Forest adds random feature selection at each split, so trees are less correlated than ordinary bagged trees. Lower correlation improves the variance-reduction effect of averaging.

A simplified split criterion for classification is Gini impurity: $G = 1 - \sum_c p_c^2$

Entropy: $H = -\sum_c p_c \log_2 p_c$

Information gain: $IG = H(\text{parent}) - \sum_j (n_j/n)H(\text{child}_j)$

Worked example. A node with 6 samples: 4 positive, 2 negative has $p_+ = 2/3$ and $p_- = 1/3$. Gini $= 1 - (4/9 + 1/9) = 4/9 \approx 0.444$. A pure node has Gini $= 0$.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=5,
    max_features="sqrt",
    bootstrap=True,
    oob_score=True,
    random_state=42
)
rf.fit(X_train, y_train)
print(rf.oob_score_)
```

9. Boosting

Boosting builds models sequentially. Each new learner is trained to improve the errors/residuals left by the current ensemble. The transcript then moves from general boosting to AdaBoost, Gradient Boosting and XGBoost. [filecite turn2file3 L60-L64](#)

9.1 Additive model

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x)$$

η is the learning rate. A small η usually requires more trees but can improve generalization by making each update conservative.

Worked example. If $F_0(x)=0.40$, $\eta=0.1$ and three weak learners output 0.5, -0.2 and 0.3, then $F_3=0.40+0.1(0.5-0.2+0.3)=0.46$.

9.2 Residual fitting for squared error

For y and current prediction $F_{m-1}(x)$, residual $r_i = y_i - F_{m-1}(x_i)$.

Fit $h_m(x) \approx r$, then update $F_m(x)=F_{m-1}(x)+\eta h_m(x)$.

Worked example. If $y=[10,14,20]$ and current prediction is $[12,12,18]$, residuals are $[-2,2,2]$. A weak tree approximating these residuals can correct the current model.

10. AdaBoost

AdaBoost focuses subsequent weak learners on examples that previous learners handled poorly. It is especially useful for understanding the general boosting idea mathematically.

10.1 Binary AdaBoost derivation

Weighted error: $\epsilon_m = \sum_i w_i \cdot 1[y_i \neq h_m(x_i)]$

Learner weight: $\alpha_m = (1/2) \ln[(1-\epsilon_m)/\epsilon_m]$

Update: $w_i \leftarrow w_i \exp(-\alpha_m y_i h_m(x_i))$, then normalize so $\sum_i w_i = 1$.

Final classifier: $H(x) = \text{sign}[\sum_m \alpha_m h_m(x)]$

Worked example. Four observations start with weights 0.25 each. If a stump misclassifies one point, $\epsilon = 0.25$. Then $\alpha = 0.5 \ln(0.75/0.25) = 0.5 \ln 3 \approx 0.549$. The misclassified point is multiplied by $e^{+0.549} \approx 1.732$; correctly classified points are multiplied by $e^{-0.549} \approx 0.577$, after which all weights are normalized.

This derivation shows the key mechanism: wrong examples gain relative weight, so later learners pay more attention to them.

11. Gradient Boosting

Gradient Boosting generalizes residual fitting: each new weak learner approximates the negative gradient of the loss with respect to the current predictions.

11.1 General derivation

Objective: $J(F) = \sum_i L(y_i, F(x_i))$.

Pseudo-residual: $r_{im} = -[\partial L(y_i, F(x_i)) / \partial F(x_i)]_{F=F_{m-1}}$.

Fit $h_m(x)$ to r_{im} , then $F_m(x) = F_{m-1}(x) + \eta h_m(x)$.

11.2 Squared-error case

$$L(y, F) = \frac{1}{2}(y - F)^2$$

$$\partial L / \partial F = F - y$$

$$-\partial L / \partial F = y - F = \text{residual}$$

So ordinary residual boosting is a special case of gradient boosting under squared-error loss.

11.3 Logistic loss

For $y \in \{0, 1\}$, $p = \sigma(F) = 1 / (1 + e^{-F})$,

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

$\partial L / \partial F = p - y$, hence pseudo-residual = $y - p$.

Worked example. If $y=1$ and current $p=0.70$, pseudo-residual=0.30. If $y=0$ and $p=0.70$, pseudo-residual=-0.70. The second point needs a much larger correction.

12. XGBoost

The source introduces XGBoost after AdaBoost and Gradient Boosting and notes that XGBoost is imported from the xgboost package rather than scikit-learn. [filecite\[turn2file10\]\[L171-L176\]](#)

XGBoost adds a more explicit regularized objective and uses a second-order Taylor approximation to choose tree updates.

12.1 Objective

$$\text{Obj}^t = \sum_i L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

$$\Omega(f) = \gamma T + (\lambda/2) \sum_{j=1}^T w_j^2$$

$$\text{Taylor approximation: } \text{Obj} \approx \text{constant} + \sum_i [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$$

$$g_i = \partial L / \partial \hat{y}_i, \quad h_i = \partial^2 L / \partial \hat{y}_i^2$$

12.2 Optimal leaf weight

For one leaf with gradient sum $G = \sum_i g_i$ and Hessian sum $H = \sum_i h_i$:

$$w^* = -G / (H + \lambda)$$

$$\text{Leaf contribution to objective} \approx -\frac{1}{2} \frac{G^2}{H + \lambda} + \gamma$$

Worked example. Suppose a leaf has $G = -4$, $H = 6$, $\lambda = 2$. Then $w^* = -(-4)/(6+2) = 0.5$. The regularizer λ shrinks the update compared with the unregularized value G/H .

```
from xgboost import XGBClassifier
```

```
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=42
)
xgb.fit(X_train, y_train)
```

13. Unsupervised Learning

Unsupervised learning works without a target label y . The source transitions from supervised ensemble learning into clustering and dimensionality reduction. [filecite turn1file5 L89-L94](#)

Main goals in this part: discover groups (clustering), detect density structure (DBSCAN), and compress features while retaining important variance (PCA).

14. K-means Clustering — Full Mathematics

K-means partitions n observations into K clusters by assigning each observation to its nearest centroid and then recomputing centroids. The source identifies K-means as the first clustering algorithm in the unsupervised section. [filecite turn2file14 L237-L242](#)

14.1 Objective function

Given assignments $c_i \in \{1, \dots, K\}$ and centroids $\mu_1, \dots, \mu_K$:

$$J(c, \mu) = \sum_{i=1}^n ||x_i - \mu_{c_i}||^2$$

K-means seeks $\operatorname{argmin}_{\{c, \mu\}} J(c, \mu)$.

14.2 Why the centroid is the mean — derivation

For one cluster C , minimize $S(\mu) = \sum_{\{x_i \in C\}} ||x_i - \mu||^2$. Differentiate with respect to μ :

$$S(\mu) = \sum_i (x_i - \mu)^T (x_i - \mu)$$

$$\partial S / \partial \mu = -2 \sum_i (x_i - \mu) = 0$$

$$\sum_i x_i - n\mu = 0 \Rightarrow \mu = (1/n) \sum_i x_i$$

Thus the arithmetic mean is not an arbitrary choice: it is the minimizer of the within-cluster squared Euclidean distance.

14.3 Worked 2D example

Point	x	y
A	1	1
B	1	2
C	2	1
D	8	8
E	9	8
F	8	9

Choose $K=2$ and initial centroids $\mu_1=(1,1)$, $\mu_2=(8,8)$. Assignment: A,B,C $\rightarrow$ cluster 1; D,E,F $\rightarrow$ cluster 2. New centroid 1 = $((1+1+2)/3, (1+2+1)/3) = (1.333, 1.333)$. New centroid 2 = $((8+9+8)/3, (8+8+9)/3) = (8.333, 8.333)$. Reassign and repeat until assignments stop changing or J stops improving.

14.4 Python

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
kmeans = KMeans(n_clusters=2, n_init=20, random_state=42)
labels = kmeans.fit_predict(X_scaled)

print(kmeans.cluster_centers_)
print(kmeans.inertia_) # within-cluster SSE
```


15. Elbow Method and Choosing K

The transcript explicitly uses the Elbow Method to decide the number of clusters K.

filecite[turn2file5[L89-L95]

$$\text{Inertia} / \text{WCSS}(K) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

As K increases, WCSS cannot increase because more centroids provide more flexibility. The useful K is often near the point where the reduction in WCSS changes from steep to gradual—the 'elbow'.

Worked example. Suppose WCSS for K=1...6 is [120, 65, 40, 34, 31, 29]. The largest gains occur from 1→2 and 2→3; after K=3, improvements are much smaller. K≈3 is a reasonable elbow candidate.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertias = []
for k in range(1, 8):
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(range(1, 8), inertias, marker="o")
plt.xlabel("K")
plt.ylabel("Within-cluster SSE / inertia")
plt.show()
```

16. Hierarchical Clustering

Hierarchical clustering builds a nested tree of clusters. In agglomerative clustering, every point begins as its own cluster and the closest pair of clusters is merged repeatedly.

16.1 Linkage mathematics

Single linkage: $d(A,B) = \min_{\{a \in A, b \in B\}} d(a,b)$

Complete linkage: $d(A,B) = \max_{\{a \in A, b \in B\}} d(a,b)$

Average linkage: $d(A,B) = (1/(|A||B|)) \sum_{a \in A} \sum_{b \in B} d(a,b)$

Worked example. $A = \{1,2\}$, $B = \{5,8\}$. With absolute distance, single linkage = $\min(4,7,3,6) = 3$; complete linkage = $\max(\dots) = 7$; average = $(4+7+3+6)/4 = 5$.

16.2 Dendrogram cut

The dendrogram records merge distances. A horizontal cut at a chosen distance produces a particular number of clusters. This differs from K-means: hierarchical clustering does not require choosing K at the beginning.

17. DBSCAN

DBSCAN is a density-based clustering method. It identifies dense regions using ϵ (eps) and MinPts (minimum number of points required in an ϵ -neighborhood).

17.1 Definitions

ϵ -neighborhood: $N_{\epsilon}(p) = \{q : d(p,q) \leq \epsilon\}$

Core point: $|N_{\epsilon}(p)| \geq \text{MinPts}$

Border point: not core, but lies within ϵ of a core point

Noise/outlier: neither core nor density-reachable from a core region

Worked example. With $\epsilon=1.5$ and $\text{MinPts}=4$, if point p has 5 neighbors within distance 1.5 (including itself, depending on implementation convention), p is core. A nearby point with only 2 neighbors can still be a border point if it lies within ϵ of a core point.

17.2 Why DBSCAN differs from K-means

- K-means optimizes squared distance to centroids and assumes roughly centroid-shaped clusters.
- DBSCAN uses density and can find non-spherical shapes.
- DBSCAN can label points as noise.
- DBSCAN does not require K in advance.

18. Dimensionality Reduction

Dimensionality reduction maps a high-dimensional representation $X \in \mathbb{R}^d$ into a lower-dimensional representation $Z \in \mathbb{R}^r$ with $r < d$. The goal is to preserve useful structure while reducing computation, noise and redundancy.

Linear projection: $z = W^T(x - \mu)$, where $W \in \mathbb{R}^{d \times r}$

PCA chooses W so that projected data has maximum variance, equivalently so that reconstruction error is minimized under an orthonormal projection.

19. PCA — Full Mathematical Derivation

The source devotes a chapter to PCA implementation after introducing dimensionality reduction.

[filecite\turn0file0\L1674-L1674](#)

19.1 Step 1 — center the data

For each feature j : $x'_{ij} = x_{ij} - \mu_j$

Centering ensures the covariance matrix describes variation around the mean rather than around the origin.

19.2 Step 2 — covariance matrix

For centered X ($n \times d$): $\Sigma = (1/(n-1)) X^T X$

$\Sigma_{\{jk\}} = \text{Cov}(X_j, X_k)$

Worked example. For two centered features, if $X_c = [[-1, -2], [0, 0], [1, 2]]$, then $X_c^T X_c = [[2, 4], [4, 8]]$. Dividing by $n-1=2$ gives $\Sigma = [[1, 2], [2, 4]]$.

19.3 Step 3 — maximize projected variance

Let w be a unit direction, $\|w\|^2=1$. Projected values are $z = X_c w$. Their variance is proportional to:

$$\text{Var}(z) = w^T \Sigma w$$

We want to maximize $w^T \Sigma w$ subject to $w^T w = 1$.

19.4 Lagrange-multiplier derivation

$$L(w, \lambda) = w^T \Sigma w - \lambda (w^T w - 1)$$

$$\partial L / \partial w = 2 \Sigma w - 2 \lambda w = 0$$

$$\Sigma w = \lambda w$$

Therefore w must be an eigenvector of the covariance matrix. The direction with the largest eigenvalue λ_1 gives the maximum variance. The second principal component is the eigenvector with the second-largest eigenvalue, subject to orthogonality, and so on.

19.5 Explained variance

Explained variance ratio for component j : $\text{EVR}_j = \lambda_j / \sum_{k=1}^d \lambda_k$

Cumulative $\text{EVR}(r) = \sum_{j=1}^r \text{EVR}_j$

Worked example. If eigenvalues are $[5, 3, 1, 1]$, total variance = 10. PC1 explains 50%, PC2 30%, PC3 10%, PC4 10%. Two components retain 80% of the variance.

19.6 Small numerical PCA example

Observation	x_1	x_2
1	1	2
2	2	4
3	3	6

Means are (2,4). Centered rows are $(-1,-2),(0,0),(1,2)$. The covariance matrix is $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. Its eigenvalues are 5 and 0. The dominant eigenvector is proportional to (1,2), so the data essentially lies on one line. One principal component captures 100% of the variance.

19.7 PCA implementation

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Components:", pca.components_)
print("Explained variance:", pca.explained_variance_)
print("Explained ratio:", pca.explained_variance_ratio_)
print("Cumulative:", pca.explained_variance_ratio_.cumsum())
```

Important: whether to standardize before PCA depends on the measurement scales. If one feature is measured in thousands and another in fractions, standardization often prevents the large-unit feature from dominating the covariance structure.

20. Integrated Worked Mini-Project

This mini-project deliberately uses a small synthetic dataset so every calculation can be inspected by hand. It ties together scaling, cross-validation, hyperparameter tuning, ensemble learning, K-means and PCA.

ID	Age	Spend	Visits	Class
A	20	20	2	0
B	22	25	3	0
C	25	30	4	0
D	45	75	8	1
E	48	80	9	1
F	50	85	10	1
G	52	90	11	1
H	28	35	5	0

20.1 Classification objective

Use Age, Spend and Visits to predict Class. A complete workflow is: split data → fit preprocessing on training only → cross-validate candidate models → tune hyperparameters → refit best model → evaluate once on a held-out test set.

20.2 Why scaling matters

For each feature: $z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$

The training mean and standard deviation must be computed inside each CV training fold, not on the entire dataset, to avoid leakage. A Pipeline is the safest implementation.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.svm import SVC

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", SVC())
])

param_grid = {
    "model__C": [0.1, 1, 10],
    "model__kernel": ["linear", "rbf"],
    "model__gamma": ["scale", 0.1]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(pipe, param_grid, cv=cv, scoring="accuracy")
search.fit(X_train, y_train)

print(search.best_params_)
print(search.best_score_)
```

20.3 Unsupervised branch

Drop Class, scale the remaining features, use K-means and inspect inertia for K=2,3,4. Then run PCA to visualize the data in two dimensions.

```
X_unsup = df[["Age", "Spend", "Visits"]]

X_scaled = StandardScaler().fit_transform(X_unsup)
```

```
for k in [2, 3, 4]:  
    km = KMeans(n_clusters=k, n_init=20, random_state=42)  
    km.fit(X_scaled)  
    print(k, km.inertia_)  
  
pca = PCA(n_components=2)  
X_2d = pca.fit_transform(X_scaled)  
print(pca.explained_variance_ratio_)
```


21. Formula Sheet and Exam Checklist

Core statistics

$$\mu = (1/n)\sum x_i$$

$$\sigma^2 = (1/n)\sum (x_i - \mu)^2$$

$$z = (x - \mu)/\sigma$$

$$d_E(x, y) = \sqrt{\sum (x_j - y_j)^2}$$

$$d_1(x, y) = \sum |x_j - y_j|$$

Cross-validation / tuning

$$CV = (1/K)\sum_k S_k$$

$$h^* = \operatorname{argmax}_h CV(h)$$

$$\text{Grid combinations} = \prod |H_j|$$

$$\text{Grid fits} \approx K \prod |H_j|$$

$$\text{Randomized fits} \approx K \cdot n_{\text{iter}}$$

Trees

$$\text{Gini} = 1 - \sum p_c^2$$

$$\text{Entropy} = -\sum p_c \log_2 p_c$$

$$IG = H(\text{parent}) - \sum (n_j/n) H(\text{child}_j)$$

Bagging / RF

$$\hat{f}(x) = (1/B)\sum \hat{f}_b(x)$$

$$P(\text{point omitted}) \approx e^{-1} \approx 0.368$$

$$\text{Expected unique bootstrap observations} \approx 0.632n$$

Boosting

$$F_M = F_0 + \sum \eta h_m$$

$$\text{AdaBoost } \alpha_m = \frac{1}{2} \ln[(1 - \epsilon_m)/\epsilon_m]$$

$$\text{Gradient pseudo-residual } r = -\partial L / \partial F$$

XGBoost

$$\text{Obj} \approx \sum [g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2] + \Omega(f)$$

$$\Omega(f) = \gamma T + (\lambda/2) \sum w_j^2$$

$$w^* = -G / (H + \lambda)$$

K-means

$$J = \sum ||x_i - \mu_{\{c_i\}}||^2$$

$$\mu_k = (1/|C_k|) \sum_{i \in C_k} x_i$$

$$WCSS(K) = J$$

DBSCAN

$$N_\epsilon(p) = \{q: d(p, q) \leq \epsilon\}$$

$$\text{Core if } |N_\epsilon(p)| \geq \text{MinPts}$$

PCA

$$\Sigma = (1/(n-1)) X_c^T X_c$$

$$\Sigma w = \lambda w$$

$$\text{EVR}_j = \lambda_j / \sum \lambda_k$$

$$z = X_c w$$

Final exam / interview checklist

- Explain the difference between a parameter and a hyperparameter.
- Derive the K-fold CV mean and explain why a single split can be misleading.
- Calculate how many GridSearchCV fits a grid requires.
- Explain why RandomizedSearchCV can be cheaper in high-dimensional spaces.
- Derive the KNN majority vote for a tiny set of distances.
- Calculate Gini impurity and entropy for a binary node.
- Explain why bootstrap sampling produces about 63.2% unique observations on average.
- Derive the AdaBoost learner weight from weighted error.
- Show why gradient boosting residuals equal negative gradients for squared loss.
- Derive the PCA eigenvalue equation using a Lagrange multiplier.
- Compute a K-means centroid by differentiating the within-cluster SSE.
- Explain DBSCAN core/border/noise points using ϵ and MinPts.
- Explain why scaling is important for KNN, SVM, K-means and often PCA.
- Explain leakage and why preprocessing should be inside a Pipeline during CV.

Source Coverage Map

The uploaded transcript is organized into chapters on model tuning, cross-validation, Grid Search CV, Random Search CV, ensemble learning, stacking, bagging, boosting, Random Forest, AdaBoost/Gradient Boost/XGBoost, unsupervised learning, K-means, DBSCAN and PCA. The chapter sequence appears explicitly in the source, including K-means, DBSCAN and PCA implementation sections.

Source chapter	Covered in this guide
2-3: model tuning	§2
4-5: cross validation + implementation	§3
6-8: hyperparameter/grid search	§2, §4
9-10: random search	§5
11-13: ensemble + stacking + bagging	§6-9
14-18: boosting + implementation	§9-12
19-21: unsupervised + K-means	§13-15
22-23: DBSCAN	§17
24-25: dimensionality reduction + PCA	§18-19

Note on mathematical supplementation: the transcript is primarily instructional and code-oriented. Where it states a concept without a full derivation, this guide derives the standard mathematical objective from first principles using small examples. Those derivations are intentionally written as a self-contained study aid rather than presented as verbatim source material.

End of guide.